1. Booking Auto-Expire / MISSED Status

Abhi booking ka time nikalne ke baad bhi database mein booking `pending` / `confirmed` / `in_progress` hi reh jaati hai. Frontend sirf UI par `Missed` show karta hai.

Backend mein:

- Booking status mein `missed` status add karo.
- Agar `no_show` already same purpose serve karta hai, to new status ki zaroorat nahi; otherwise `missed` use karo.
- Scheduled cron/job **har 5–10 minutes** mein run ho.
- Cron `pending`, `confirmed`, aur `in_progress` bookings check kare.
- Expiry formula:

```
expiresAt = bookingDate + startTime + 2 hours
```

Agar:

```
currentDateTime > expiresAt
```

to booking ka actual DB status permanently:

```
missed
```

kar do.

Example:

Booking:

- Date: 08-09-2026
- Start: 08:00 AM
- Grace: 2 hours
- Expiry: 10:00 AM

10:00 AM ke baad agar booking completed/cancelled nahi hai, to:

```
status = missed
```

hona chahiye.

Isse customer app, artist app, reports aur database sab mein same status rahega.

2. Expired Booking ko API se bhi Update Hone Se Rokna Hai

Endpoint:

`PUT /artists/bookings/update_status/`

Sirf frontend button hide karna enough nahi hai.

Backend par bhi validation lagao.

Agar booking ka `expiresAt` cross ho chuka hai, to:

- Accept ❌
- Confirm ❌
- On-My-Way ❌
- In-Progress ❌
- Any other active status change ❌

allow nahi hona chahiye.

Example response:

```
{
  "success": false,
  "message": "This booking has expired and cannot be updated."
}
```

HTTP status preferably `400` (ya project ke existing validation convention ke according appropriate 4xx).

Important: Direct API call karke koi expired booking ko accept/confirm nahi kar paana chahiye.

3. Status Rules Backend Authoritative Hone Chahiye

Final booking statuses:

- `pending`
- `confirmed`
- `in_progress`
- `completed`
- `cancelled`
- `missed`

Status logic:

Cancelled → CANCELLED
Completed → COMPLETED

Expiry cross ho gayi + not completed/cancelled → MISSED
Future valid booking → UPCOMING / existing active status

Expired booking ko dobara `confirmed`, `accepted`, `on_the_way` ya `in_progress` nahi kiya ja sake.

Past booking ko frontend mein Upcoming nahi dikhna chahiye, aur backend response bhi stale active status par depend nahi karna chahiye.

Agar possible ho to API response mein:

```
{
  "status": "confirmed",
  "derivedStatus": "missed"
}
```

jaisa `derivedStatus` bhi bhej sakte ho, taaki Customer aur Artist dono apps mein alag-alag status calculation na likhni pade.

Lekin `derivedStatus` actual DB status ka replacement nahi hai — **cron ko actual DB status bhi `missed` karna hai.**

4. Timezone Confirm / Same Reference Use Karo

Please confirm karo ki:

`bookingDate + startTime`

database/API mein:

- UTC mein store/compare ho raha hai?
- Ya Artist ke local timezone mein?

Cron, API validation aur frontend ko same timezone reference use karna hai.

Agar project UTC use karta hai to expiry calculation bhi UTC-aware honi chahiye aur frontend par local time display ho.

Existing implementation check karke hi timezone change karna, bina check kiye existing data ko convert mat karna.

5. expiresAt

Abhi `expiresAt` sirf new pending requests ke case mein aa raha hai.

Confirmed aur in-progress bookings ke liye bhi `expiresAt` return karo, ya backend par same formula se calculate karo:

```
expiresAt = bookingDate + startTime + 2 hours
```

Ideally API response mein expiry clear honi chahiye:

```
{
  "bookingDate": "2026-09-08",
  "startTime": "08:00",
  "expiresAt": "2026-09-08T10:00:00"
}
```

Cron aur `update_status` API dono isi same expiry rule ko follow karein.

6. Same-Day Booking + 2 Hour Rule Backend Par Validate Karo

Same-day booking allowed hai, **lekin current time se minimum 2 hours baad ka slot hi book ho sakta hai.**

Example:

Current time = 2:00 PM
Earliest booking = 4:00 PM

Current time = 5:00 PM
Earliest booking = 7:00 PM

Backend validation:

```
bookingDateTime >= currentDateTime + 2 hours
```

Frontend validation ke saath backend par bhi enforce hona chahiye.

7. Past Date Booking Block

Past dates book nahi ho sakti.

Example:

Yesterday → ❌
Today → ✅ only if minimum 2-hour rule satisfy hota hai
Tomorrow → ✅

Backend ko bhi past date reject karni hai.

8. Slot Clash / Double Booking

Slot clash, working-hours aur availability check **abhi sahi kaam kar rahe hain**, inmein unnecessary changes nahi chahiye.

Bas backend par race-condition protection ensure karna hai, taaki same slot ko exactly same time par 2 customers book na kar saken.

Agar ek artist ka slot:

`7:00 PM – 9:00 PM`

booked hai, to overlapping:

`7:30`, `8:00`, `8:30`

etc. unavailable hone chahiye.

9. Payment Type – Valid Values Confirm Karo

Currently testing mein:

`full` → works

Lekin:

`upi` / `card` → reject ho rahe hain.

Please backend se exact valid payment-type list confirm kar do.

Agar possible ho to ek small API bhi provide kar sakte ho:

`GET /bookings/payment-types/`

Example:

```
{
  "success": true,
  "paymentTypes": [
    "full",
    "upi",
    "card"
  ]
}
```

Actual valid values backend ke existing enum/model ke according confirm karna hai; upar wali list example hai.

10. Multiple Services / Packages

Customer ek booking mein multiple packages/services aur multiple quantities/persons book kar sakta hai.

Example:

Party Makeup × 3 = ₹1500
Bridal Makeup × 1 = ₹3000
Total = ₹4500

One-package-at-a-time restriction nahi honi chahiye.

Lekin currently agar har item/package ka payment separate ja raha hai aur ye business requirement ke according correct hai, to is point par backend change required nahi hai.

11. "Conversation Not Found" – Normal Empty State

Agar customer/artist ne kabhi support chat start hi nahi ki hai, to:

`400 Conversation not found`

return nahi hona chahiye.

Ye normal empty state hai.

Instead:

HTTP `200`

```
{
  "success": true,
  "data": []
}
```

return karo.

Frontend isko:

`No conversation yet`

show kar sakta hai.

Isse unnecessary 400 errors/logs bhi avoid honge.

12. Inactive / Disabled Package ka Clear Error

Agar package inactive/band hai aur customer usko book karne ki koshish karta hai, to generic confusing error ke bajay clear message bhejo:

```
{
  "success": false,
  "message": "This package is currently unavailable."
}
```

13. Date Format

Standard date format ideally:

`YYYY-MM-DD`

Example:

`2026-09-08`

Current `DD-MM-YY` agar existing API mein use ho raha hai to abhi breaking change zaroori nahi hai.

Either future mein standard format use kar dena ya API documentation mein clearly mention kar dena ki exact expected format kya hai

14. Payment Success ≠ Service Completed

Payment status aur booking/service status separate hone chahiye.

Example:

payment_status = paid
booking_status = confirmed/upcoming

Payment successful hone ka matlab service completed nahi hai.

Service complete hone par hi:

```
booking_status = completed
```

hona chahiye.

Agar expiry cross ho gayi aur service complete nahi hui:

```
booking_status = missed
```

hona chahiye.

15. Backend Total/Price Validate Kare

Frontend se aaye hue amount ko blindly trust nahi karna.

Backend:

- Package DB se verify kare
- Package artist ka hi hai verify kare
- Unit price DB se le
- Quantity validate kare
- Subtotal calculate kare
- Final total server-side calculate kare

Example:

```
₹500 × quantity 3 = ₹1500
```

Frontend ka manipulated total accept nahi hona chahiye.

16. Payment Verification

Payment success sirf frontend response ke basis par booking ko paid/confirmed mat karo.

Payment gateway ke server-side verification/webhook ke baad hi payment ko verified/paid mark karo.

Uske baad booking confirmation/receipt generate ho.

17. Booking Flow Final

Customer side:

Artist
↓
Packages/Services
↓
Multiple Packages + Quantity
↓
Cart
↓
Location
↓
Date
↓
Available Time Slot
↓
Backend Validation
↓
Payment
↓
Payment Verification
↓
Booking Confirmed
↓
Booking Detail / Receipt

Receipt mein ideally:

- Booking ID
- Customer
- Artist
- Package/service
- Quantity/persons
- Price
- Total
- Date
- Start/end time
- Location
- Payment ID/status
- Booking status

hona chahiye.

18. Artist Side Status/Actions

Artist app mein:

Future booking → appropriate active action (Accept/Confirm etc.)

Expired booking → `Missed`

Expired booking par:

`Accept` ❌
`Confirm` ❌
`On-My-Way` ❌
`In-Progress` ❌

Service actually complete hone par:

`Completed`

Past uncompleted booking ko automatically:

`Missed`

hona chahiye.

19. Final Priority

MUST FIX

1. Auto-expire cron
2. `missed` DB status
3. Expired booking API validation
4. Timezone confirmation
5. `expiresAt`
6. Backend +2 hour validation
7. Past-date validation
8. Payment verification
9. Server-side price/total validation
10. Race-condition/double-booking protection

SHOULD FIX

11. Payment type list confirmation/API
12. Conversation not found → 200 empty data
13. Clear inactive package error
14. Standard date format/documentation
15. Optional `derivedStatus`

ALREADY WORKING — NO UNNECESSARY CHANGE

16. Slot clash
17. Working-hours check
18. Availability data

Main requirement ye hai ki **Customer App, Artist App, Database aur Reports sab mein booking status ka single consistent source of truth ho.**

Especially:

Booking ka time + 2 hours grace period cross hone ke baad backend khud database mein `missed` status set kare, aur uske baad kisi bhi direct API call se booking ko active status mein update na kiya ja sake